



Let's Program Live

Joy and Pain of Concurrency With C++11 and C++20

Nico Josuttis

2024

07/24

C++

©2024 by josuttis.com

1

josuttis | eckstein
IT communication

Nicolai M. Josuttis

- **Independent consultant**
 - Continuously learning since 1962
- **C++:**
 - since 1990
 - ISO Standard Committee since 1997
- **Other Topics:**
 - Systems Architect
 - Technical Manager
 - SOA
 - X and OSF/Motif



C++

©2024 by iosuttis.com

communication

Modern C++

Starting Threads

C++

©2024 by josuttis.com

3

josuttis | eckstein
IT communication

Starting a Thread with `std::jthread`

C++11

- `std::async()` and `std::future<>`
 - High-level API
 - Function that requests to start a function/lambda in its own thread
 - Returns a handle to yield the outcome (return value or exception)
 - Request may be deferred until outcome is needed
- `std::thread`
 - Low-level API
 - Type of **object** that start a function/lambda in its own thread
 - Constructor with **callable** starts the thread
 - Has to **join()** for (or **detach()**) the thread before destructor is called
 - Threads can't return anything
 - Threads are not allowed to end with an exception

Use `std::jthread` since C++20**C++**

©2024 by josuttis.com

4

josuttis | eckstein
IT communication

Using Class `std::thread`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t(task, name, 42);
    ...
}
```

s refers to a **copy** of name
(even as a reference)

```
void task(const std::string& s,
          int val)
{
    ...
}
```

C++

©2024 by josuttis.com

5

josuttis | eckstein
IT communicationUsing Class `std::thread` and `std::cref()`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t(task, std::cref(name), 42);
    ...
}
```

s **refers** to name
(as a reference)

```
void task(const std::string& s,
          int val)
{
    ...
}
```

C++

©2024 by josuttis.com

6

josuttis | eckstein
IT communication

Using Class `std::thread`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t{task, name, val};
    ...
}
```

```
void task(const std::string& s,
          int val)
{
    ...
}
```

On thread end with exception
`std::terminate() => std::abort()`

C++

©2024 by josuttis.com

7

josuttis | eckstein
IT communicationUsing Class `std::thread`

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t{task, name, val};
    ...
}
```

```
void task(const std::string& s,
          int val)
{
    try {
        ...
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

8

josuttis | eckstein
IT communication

Using Class `std::thread`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t{task, name, val};
    ...
}
```

There is a bug

(unless core dumps don't matter)

```
void task(const std::string& s,
          int val)
{
    try {
        ...
    }
    catch (...) {
        ... // handle every exception
    }
}
```

Destructor of a `std::thread` will
`std::terminate() => std::abort()`
 unless `join()` or `detach()` was called

C++

©2024 by josuttis.com

9

josuttis | eckstein

IT communication

Using Class `std::thread`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t{task, name, val};
    ...

    // wait for thread to finish:
    t.join();
    ...
}
```

There is still a bug

(unless core dumps don't matter)

On exception, destructor will still
`std::terminate() => core dump`
 because `join()` was not called

```
task(const std::string& s,
      int val)
{
    try {
        ...
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

10

josuttis | eckstein

IT communication

Using Class `std::thread`

C++11

```
#include <thread>

void foo()
{
    std::string name = "Kim";
    ...
    // start thread calling task() with two args:
    std::thread t{task, name, val};
    try {
        ...
    }
    catch (...) { // on exception
        ...
        t.join(); // wait for thread
        throw;    // rethrow exception
    }
    // wait for thread to finish:
    t.join();
    ...
}
```

```
void task(const std::string& s,
          int val)
{
    try {
        ...
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

11

josuttis | eckstein
IT communicationUsing Class `std::thread` for **Two** Threads

C++11

```
...
std::thread t1{task1, name, val};

try {
    std::thread t2{task2, name, val};
    ...
}
catch (...) { // on exception
    ...      // - signal stop

    t1.join(); // - wait for end of task1
    throw;
}
t1.join();
t2.join();    // ERROR: t2 is out of scope
...
```

Problem with two threads:

- Has to start `t2` in `try` scope because that might throw
- But need to join `t2` in same scope as `t1`

C++

©2024 by josuttis.com

12

josuttis | eckstein
IT communication

Using Class `std::thread` for **Two** Threads

C++11

```

...
std::thread t1{task1, name, val};
std::thread t2;
try {
    t2 = std::thread{task2, name, val};
    ...
}
catch (...) { // on exception
    ... // - signal stop
    if (t2.joinable()) { // if started
        t2.join(); // - wait for end of task2
    }
    t1.join(); // - wait for end of task1
    throw;
}
t1.join();
t2.join(); // OK
...

```

C++

©2024 by josuttis.com

13

josuttis | eckstein
IT communicationUsing Class `std::thread` for **Two** Threads

C++11/C++20

```

...
std::thread t1{task1, name, val};
std::thread t2;
try {
    t2 = std::thread{task2, name, val};
    ...
}
catch (...) { // on exception
    ... // - signal stop
    if (t2.joinable()) { // if started
        t2.join(); // - wait for end of task2
    }
    t1.join(); // - wait for end of task1
    throw;
}
t1.join();
t2.join(); // OK
...

```

// since C++20 just:

```

...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
t1.join();
t2.join();
...

```



Prefer `std::jthread` over `std::thread`
(same header, same API)

C++

©2024 by josuttis.com

14

josuttis | eckstein
IT communication

Using Class `std::jthread`

C++20

```
...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
```

On exception:

```
if (t2.joinable()) {
    t2.request_stop();
    t2.join();
} // Destructor t2
```

```
if (t1.joinable()) {
    t1.request_stop();
    t1.join();
} // Destructor t1
```

```
t1.join();
t2.join();
...
```

```
void task1(const std::string& n, int val)
{
    try {
        ... // stop request ignored
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

15

josuttis | eckstein
IT communicationUsing Class `std::jthread` and Stop Tokens

C++20

```
...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
```

On exception:

```
if (t2.joinable()) {
    t2.request_stop();
    t2.join();
} // Destructor t2
```

```
if (t1.joinable()) {
    t1.request_stop();
    t1.join();
} // Destructor t1
```

```
t1.join();
t2.join();
...
```

optional 1st parameter

```
void task1(std::stop_token st,
           const std::string& n, int val)
{
    try {
        ...
        if (st.stop_requested()) {
            return;
        }
        ...
        while (!st.stop_requested()) {
            ...
        }
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

16

josuttis | eckstein
IT communication

Using Class `std::jthread` and Stop Tokens

C++20

```
...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
```

```
if (noLongerNeedTask1) {
    // signal first thread to stop:
    t1.request_stop();
}
...
```

```
t1.join();
t2.join();
...
```

optional 1st parameter

```
void task1(std::stop_token st,
           const std::string& n, int val)
{
    try {
        ...
        if (st.stop_requested()) {
            return;
        }
        ...
        while (!st.stop_requested()) {
            ...
        }
    }
    catch (...) {
        ... // handle every exception
    }
}
```

C++

©2024 by josuttis.com

17

josuttis | eckstein
IT communicationUsing Class `std::jthread` and Stop Callbacks

C++20

```
...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
```

```
if (noLongerNeedTask1) {
    // signal first thread to stop:
    t1.request_stop();
}
...
```

```
t1.join();
t2.join();
...
```

optional 1st parameter

```
void task1(std::stop_token st,
           const std::string& n, int val)
{
    try {
        ...
        // register callback to be called on stop request:
        std::stop_callback cb{st,
                               []{ ... } };
        ...
        while (!st.stop_requested()) {
            ...
        }
    }
    catch (...) {
        ... // handle every exception
    }
}
```

callback called
within the thread
requesting the stop

C++

©2024 by josuttis.com

18

josuttis | eckstein
IT communication

Modern C++

Dealing with Concurrency

C++

©2024 by josuttis.com

19

josuttis | eckstein
IT communication

Dealing with Concurrent Access

C++11 / C++20

- A couple of **problems** can occur:
 - Concurrent access
 - Half-written memory
 - Out-of-order updates
- Support for
 - **Mutexes**
 - Grant for exclusive or shared access
 - **Atomics**
 - Avoid half-written data and out-of-order updates
 - Includes wait and notification mechanism (since C++20)
 - **Condition variables**
 - Wait and notification mechanism
 - **Semaphores, Latches, Barriers** (since C++20)
 - **Synchronized Streams** (since C++20)

C++

©2024 by josuttis.com

20

josuttis | eckstein
IT communication

The Problems of Concurrent Data Access

C++11

• Unsynchronized data access

- C++ standard library functions usually don't support concurrent writes or reads concurrently performed with writes to the same data

```
std::vector<int> v;

if (!v.empty()) {
    cout << v.front();
}

v = ...
```

Mutexes

solve this problem

• Half-written data

```
int x = 0;

x = -1;

cout << x;
```

Atoms and Mutexes

solve this problem

might output: 0, -1, or any other value

• Out-of-order Updates

```
long data;
bool ready = false;

data = 42;
ready = true;

while (!ready) {
    ...
}
cout << data;
```

Memory barriers / Fences

(in atomics) solve this problem

might call `cout << data`**before**42 was assigned to `data`

(reordered updates from thread to thread)

C++

©2024 by josuttis.com

21

josuttis | eckstein

IT communication

Modern C++

Mutexes

C++

©2024 by josuttis.com

22

josuttis | eckstein

IT communication

Mutex and Guard Classes

C++11 / C++14 / C++17

Use the weakest mutex that fits all needs
(most likely has native OS support)

• Mutex classes:

- Simple mutex: `std::mutex`
- Nested mutexes: `std::recursive_mutex`
- Simple mutex with timeout: `std::timed_mutex`
- Nested mutex with timeout: `std::recursive_timed_mutex`
- Read/Write mutex with timeout: `std::shared_timed_mutex` (since C++14)
- Read/Write mutex: `std::shared_mutex` (since C++17)

• Lock guards:

- Trivial exclusive lock guard for 1 mutex: `std::lock_guard`
- Extended exclusive lock guard for 1 mutex: `std::unique_lock`
+ Members `lock()`, `unlock()`, `try_lock...`(), `owns_lock()`, `release()`, ...
- Shared lock for 1 mutex: `std::shared_lock` (since C++14)
- Exclusive lock for **multiple** mutexes: `std::scoped_lock` (since C++17)

C++

©2024 by josuttis.com

23

josuttis | eckstein
IT communication

Mutex Classes

C++11 / C++14 / C++17

Use the weakest mutex
that fits all needs

	mutex	recursive_mutex	shared_mutex C++17	shared_timed_mutex C++14	timed_mutex	recursive_timed_mutex
<code>lock()</code>	Acquires an exclusive lock (blocks until available)					
<code>try_lock()</code>	Tries to acquire an exclusive lock (returns <code>false</code> if not available)					
<code>unlock()</code>	Release a lock (acquired by the same thread)					
<code>try_lock_for()</code>	-	-	-	Tries for a duration of time to acquire an exclusive lock		
<code>try_lock_until()</code>	-	-	-	Tries until a certain timepoint to acquire an exclusive lock		
<code>lock_shared()</code>	-	-	Acquires a shared lock		-	-
<code>try_lock_shared()</code>	-	-	Tries to acquire a shared lock			
<code>unlock_shared()</code>	-	-	Release a shared lock		-	-
<code>try_lock_shared_for()</code>	-	-	-	Yes	-	-
<code>try_lock_shared_until()</code>	-	-	-	Yes	-	-
Multiple locks by same thread	-	Yes	-	-	-	Yes

C++

©2024 by josuttis.com

24

josuttis | eckstein
IT communication

C++14: Using Shared Locks

C++14

```
#include <shared_mutex>
#include <mutex>

...
std::vector<double> values{0, 8, 15}; // shared resource
std::shared_timed_mutex valuesMutex; // control access to v
```

```
void printAll()
{
    // lock for shared read access:
    std::shared_lock<std::shared_timed_mutex> sl{valuesMutex};
    for (double d : values) {
        std::cout << d << ' ';
    }
    std::cout << '\n';
}
```

- Constructor calls `lock_shared()`
- Destructor calls `unlock_shared()`

```
double getLast()
{ // lock for shared read access:
    std::shared_lock<std::shared_timed_mutex> sl{valuesMutex};
    return values.empty() ? 0 : values.back();
}
```

```
void insert(double val)
{ // lock for exclusive write access:
    std::lock_guard<std::shared_timed_mutex> lg{valuesMutex};
    values.push_back(val);
}
```

- Constructor calls `lock()`
- Destructor calls `unlock()`

C++

©2024 by josuttis.com

25

josuttis | eckstein
IT communication

C++17: Using Shared Locks and CTAD

C++17

```
#include <shared_mutex>
#include <mutex>

...
std::vector<double> values{0, 8, 15}; // shared resource
std::shared_mutex valuesMutex; // control access to v
```

```
void printAll()
{
    // lock for shared read access:
    std::shared_lock sl{valuesMutex};
    for (double d : values) {
        std::cout << d << ' ';
    }
    std::cout << '\n';
}
```

`shared_mutex` instead of
`shared_timed_mutex`

```
double getLast()
{ // lock for shared read access:
    std::shared_lock sl{valuesMutex};
    return values.empty() ? 0 : values.back();
}
```

No need to pass mutex type
to lock guard declarations
due to **CTAD**

```
void insert(double val)
{ // lock for exclusive write access:
    std::scoped_lock lg{valuesMutex};
    values.push_back(val);
}
```

`scoped_lock` instead of
`lock_guard`

C++

©2024 by josuttis.com

26

josuttis | eckstein
IT communication

Modern C++

Atomics

C++

©2024 by josuttis.com

27

josuttis | eckstein
IT communication

Using Atomics

C++11

- **Atomics** synchronize concurrent access
 - For individual statements
 - To avoid half-written data

```
#include <mutex>
...
std::atomic<bool> done{false};
std::mutex doneMx;
```

```
void thread1()
{
    // do something thread2 waits for
    ...
    done.store(true); // or: done = true
    ...
}
```

Instead of:

```
{
    std::lock_guard lg{doneMx};
    done = true;
} // release lock
```

```
void thread2()
{
    // wait until done flag is true (thread1 is done)
    while (!done.load()) { // or: while(!done)
        ... // do something else or sleep
    }
    ... // deal with done
}
```

Instead of:

```
{
    std::unique_lock lg{doneMx};
    while (!done) {
        lg.unlock();
        ... // do something else or sleep
        lg.lock();
    }
} // release lock
```

C++

©2024 by josuttis.com

28

josuttis | eckstein
IT communication

Using Atomics

C++11

- **Atomics** synchronize concurrent access
 - For individual statements
 - To avoid half-written data

Always have to get initialized
(otherwise undefined behavior)

```
#include <mutex>
...
std::atomic<bool> done{false};
```

```
void thread1()
{
    // do something thread2 waits for
    ...
    done.store(true); // or: done = true
    ...
}
```

```
void thread2()
{
    // wait until done flag is true (thread1 is done)
    while (!done.load()) { // or: while(!done)
        ... // do something else or sleep
    }
    ... // deal with what thread1 has prepared
}
```

C++

©2024 by josuttis.com

29

josuttis | eckstein
IT communication

Atomics for Trivially-Copyable Types

C++11

```
#include <atomic>
...
struct MyStruct {
    long id;
    std::array<double, 10> values{};
    int size = 0;
    MyStruct(int i)
        : id{i} {
    }
};

std::atomic<MyStruct> data{MyStruct{42}};
```

Trivially copyable:

- No user-defined default constructor, copy constructor, assignment, destructor
- No virtual
- Only trivially copyable members
 - No std::string, std::vector, ...

```
void thread1()
{
    MyStruct ms{11};
    ...
    ms.values[ms.size++] = x;
    ...
    // update value of data:
    data.store(ms);
}
```

```
void thread2()
{
    ...
    // process current value of data:
    MyStruct val = data.load();
    ...
}
```

C++

©2024 by josuttis.com

30

josuttis | eckstein
IT communication

Modern C++

Condition Variables

C++

©2024 by josuttis.com

31

josuttis | eckstein
IT communication

Motivation for Condition Variables

C++11

- Active polling is not the best:

```
#include <mutex>
...
bool done = false;
std::mutex doneMx;
```

```
void thread1()
{
    // do something thread2 waits for
    ...
    {
        std::lock_guard<std::mutex> lg{doneMx};
        done = true;
    } // release lock
    ...
}
```

```
void thread2()
{
    // wait until done flag is true (thread1 is done)
    {
        std::unique_lock<std::mutex> lg{doneMx};
        while (!done) {
            lg.unlock();
            ... // do something else or sleep
            lg.lock();
        }
    } // release lock

    ... // deal with what thread1 has prepared
}
```

Tradeoff between

- Using the CPU
- Delay of reaction

C++

©2024 by josuttis.com

32

josuttis | eckstein
IT communication

Condition Variables

C++11

- Let the platform signal readiness
 - Avoids active polling

```
#include <condition_variable>
...
std::condition_variable readyCV;

void thread1()
{
    ...
    // wake up one waiting thread:
    readyCV.notify_one();

    ...
    // wake up all waiting threads:
    readyCV.notify_all();
}

void thread2()
{
    ...
    // wait until woken up
    readyCV.wait(...);
    ...
}

void thread3()
{
    ...
    // wait a period of time or until woken up
    readyCV.wait_for(...);
    ...
}
```

- BUT: Beware of "spurious wakeups"

C++

©2024 by josuttis.com

33

 josuttis | eckstein
IT communication

Dealing with Spurious Wakeups

C++11

- Wrong:

```
std::vector<int> data;
std::condition_variable dataCV;

...
data.push_back(value);
dataCV.notify_one();

...
dataCV.wait();
process(data.back());
```

There might be no value from push_back()

- Right:

```
std::vector<int> data;
std::mutex dataMx;
std::condition_variable dataCV;

...
{
    std::lock_guard lg{dataMx};
    data.push_back(value);
    dataCV.notify_one();
} // release lock

...
{
    std::unique_lock lg{dataMx};
    while (data.empty()) {
        dataCV.wait(lg);
    }
    process(data.back());
} // release lock
```

might wake up more than one waiting thread

wait():
lg.unlock() and block
... (sleep)
on wakeup: lg.lock()

C++

©2024 by josuttis.com

34

 josuttis | eckstein
IT communication

Dealing with Spurious Wakeups Inside `wait()`

C++11

• **Wrong:**

```
std::vector<int> data;
std::condition_variable dataCV;

...
data.push_back(value);
dataCV.notify_one();

...
dataCV.wait();
process(data.back());
```

There might be no value from `push_back()`

• **Right:**

```
std::vector<int> data;
std::mutex dataMx;
std::condition_variable dataCV;

...
{
    std::lock_guard lg{dataMx};
    data.push_back(value);
    dataCV.notify_one();
} // release lock
```

might wake up more than one waiting thread

```
...
{
    std::unique_lock lg{dataMx};
    dataCV.wait(lg, [&]{
        return !data.empty();
    });
    process(data.back());
} // release lock
```

`wait()` handles spurious wakeups

`wait()` loops over: `lg.unlock()` and `block` ... (sleep) on wakeup: `lg.lock()`

C++

©2024 by josuttis.com

35

josuttis | eckstein
IT communication

Stop Tokens and Condition Variables

C++20

```
...
std::jthread t1{task1, name, val};
std::jthread t2{task2, name, val};
...
```

```
t1.join();
t2.join();
...
```

```
void task1(std::stop_token st,
           const std::string& n, int val)
{
    try {
        ...
        // interruptible wait:
        std::unique_lock lg{readyMx};
        if (!readyCV.wait(lg, st, ...)) {
            // stop requested:
            return;
        }
        ...
    }
    catch (...) {
        ... // handle every exception
    }
}
```

Condition variable `readyCV` has to have type `std::condition_variable_any`

C++

©2024 by josuttis.com

36

josuttis | eckstein
IT communication

Thank You!



Nicolai M. Josuttis

www.josuttis.com

nico@josuttis.com

[@NicoJosuttis](https://twitter.com/NicoJosuttis)



C++

©2024 by josuttis.com

37

josuttis | eckstein
IT communication